

Tarea 4 – Simulación estocástica

Lenin Pavón

2026-06-02

Para empezar importemos las siguientes bibliotecas:

- `tidyverse`
- `quantmod`. Para importación de datos de Yahoo Finance.
- `rugarch`. Para creación de los modelos

Ahora cargaremos 2 activos,

```
ticker <- c("MSFT", "AAPL")
getSymbols(
  Symbols = ticker,
  src = "yahoo",
  from = "2025-01-01",
  auto.assign = TRUE
)
```

```
## [1] "MSFT" "AAPL"
```

Tenemos que ajustar los precios para ajustar por *stock split* (cuando una acción se divide en más) y pago de dividendos, para lo que usamos la función `Ad`. Posteriormente, los juntamos en un solo dataframe. Limitaremos a los

```
p <- as.numeric(MSFT$"MSFT.Adjusted")
q <- as.numeric(AAPL$"AAPL.Adjusted")
# Precios ajustados
n <- 200
precios <- data.frame(
  MSFT = p,
  AAPL = q
)
# Convertir a vector numérico
# colnames(precios) <- ticker
head(precios)
```

```
##      MSFT      AAPL
## 1 413.6726 242.3019
## 2 418.3867 241.8150
## 3 422.8339 243.4446
## 4 417.4182 240.6723
## 5 419.5826 241.1592
## 6 414.0383 235.3464
```

Ejercicio 1

Haga una prueba Ljung Box con lags 5, 10 y 15 individualmente para ver si los datos pueden considerarse como una muestra, posteriormente considere los rendimientos logarítmicos y adicionalmente a la prueba Ljung Box haga una prueba Akaike y grafique las funciones de autocorrelación para los lags antes mencionados.

Prueba Ljung-Box

Realizaremos esta prueba de autocorrelación para determinar si tenemos una muestra. Definimos los lags a los que realizaremos la prueba

```
lags <- c(5, 10, 15)
```

Realizamos la prueba

```
for (name in colnames(precios)){
  for (lag in lags) {
    res <- Box.test(
      precios[name],
      lag = lag,
      type = "Ljung-Box"
    )
    cat("asset: ", name, " | lag: ", lag, "\n")
    print(res)
  }
}
```

```
## asset: MSFT | lag: 5
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 1672.6, df = 5, p-value < 2.2e-16
##
## asset: MSFT | lag: 10
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 3198.5, df = 10, p-value < 2.2e-16
##
## asset: MSFT | lag: 15
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 4573.2, df = 15, p-value < 2.2e-16
##
## asset: AAPL | lag: 5
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 1595.8, df = 5, p-value < 2.2e-16
##
```

```
## asset: AAPL | lag: 10
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 2944.5, df = 10, p-value < 2.2e-16
##
## asset: AAPL | lag: 15
##
## Box-Ljung test
##
## data: precios[name]
## X-squared = 4132.3, df = 15, p-value < 2.2e-16
```

Observemos que se rechaza la hipótesis de que los precios se comportan como una muestra aleatoria. De estas pruebas, podemos proceder con el resto del problema.

Rendimientos logarítmicos

Calculemos los rendimientos logarítmicos y eliminemos los la primera fila nula

```
ret_MSFT <- na.omit(diff(log(p)))
ret_AAPL <- na.omit(diff(log(q)))

ret <- data.frame(
  MSFT = ret_MSFT,
  AAPL = ret_AAPL
)
# colnames(ret) <- ticker
head(ret)
```

```
##           MSFT           AAPL
## 1  0.011331238 -0.002011468
## 2  0.010573316  0.006716400
## 3 -0.012890858 -0.011453051
## 4  0.005171661  0.002020819
## 5 -0.013301882 -0.024398977
## 6 -0.004209713 -0.010398046
```

Prueba Ljung-Box

Realizamos las mismas pruebas de Ljung-Box pero ahora sobre los rendimientos

```
for (name in colnames(ret)){
  for (lag in lags) {
    res <- Box.test(
      ret[name],
      lag = lag,
      type = "Ljung-Box"
    )
    cat("asset: ", name, " | lag: ", lag, "\n")
    print(res)
  }
}
```

```
## asset: MSFT | lag: 5
##
```

```
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 2.6526, df = 5, p-value = 0.7534
##
## asset:  MSFT | lag: 10
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 4.9977, df = 10, p-value = 0.8913
##
## asset:  MSFT | lag: 15
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 13.603, df = 15, p-value = 0.5558
##
## asset:  AAPL | lag: 5
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 12.145, df = 5, p-value = 0.03285
##
## asset:  AAPL | lag: 10
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 16.341, df = 10, p-value = 0.09027
##
## asset:  AAPL | lag: 15
##
## Box-Ljung test
##
## data:  ret[name]
## X-squared = 18.856, df = 15, p-value = 0.2203
```

De estos resultados vemos que sí se comporta como una muestra aleatoria.

Prueba Akaike

Definimos el arreglo dónde guardaremos los valores de las pruebas de Akaike.

```
aic_vals <- numeric(30)
```

Y realizamos la prueba para cada uno de estos valores

```
for (i in 1:15) {
  res <- arima(
    precios[name],
    order = c(i, 0, 0)
  )
  aic_vals[i] <- AIC(res)
```

```

}

## Warning in arima(precios[name], order = c(i, 0, 0)): possible convergence
## problem: optim gave code = 1

for (i in 16:30) {
  res <- arima(
    precios[name],
    order = c(i - 15, 0, 0)
  )
  aic_vals[i] <- AIC(res)
}

## Warning in arima(precios[name], order = c(i - 15, 0, 0)): possible convergence
## problem: optim gave code = 1

```

Organizando los datos de una forma tabular,

```

tabla_aic <- data.frame(
  idx = 1:15,
  MSFT = round(aic_vals[ 1:15], 4),
  AAPL = round(aic_vals[16:30], 4)
)
print(tabla_aic)

```

```

##      idx      MSFT      AAPL
## 1      1 2024.590 2024.590
## 2      2 2025.304 2025.304
## 3      3 2024.805 2024.805
## 4      4 2026.553 2026.553
## 5      5 2025.646 2025.646
## 6      6 2027.543 2027.543
## 7      7 2029.392 2029.392
## 8      8 2029.191 2029.191
## 9      9 2029.973 2029.973
## 10     10 2031.777 2031.777
## 11     11 2033.766 2033.766
## 12     12 2035.645 2035.645
## 13     13 2037.464 2037.464
## 14     14 2038.995 2038.995
## 15     15 2040.349 2040.349

```

Ahora veamos cual es el mejor orden para cada activo,

```

best_MSFT <- which.min(aic_vals[ 1:15])
best_AAPL <- which.min(aic_vals[16:30])

cat(
  "BestMejor modelo según AIC:\nMSFT: AR(",
  best_MSFT,
  ")\nAAPL: AR(",
  best_AAPL,
  ")\n",
  sep = ""
)

```

```

## BestMejor modelo según AIC:

```

```
## MSFT: AR(1)
## AAPL: AR(1)
```

Gráficas de autocorrelación

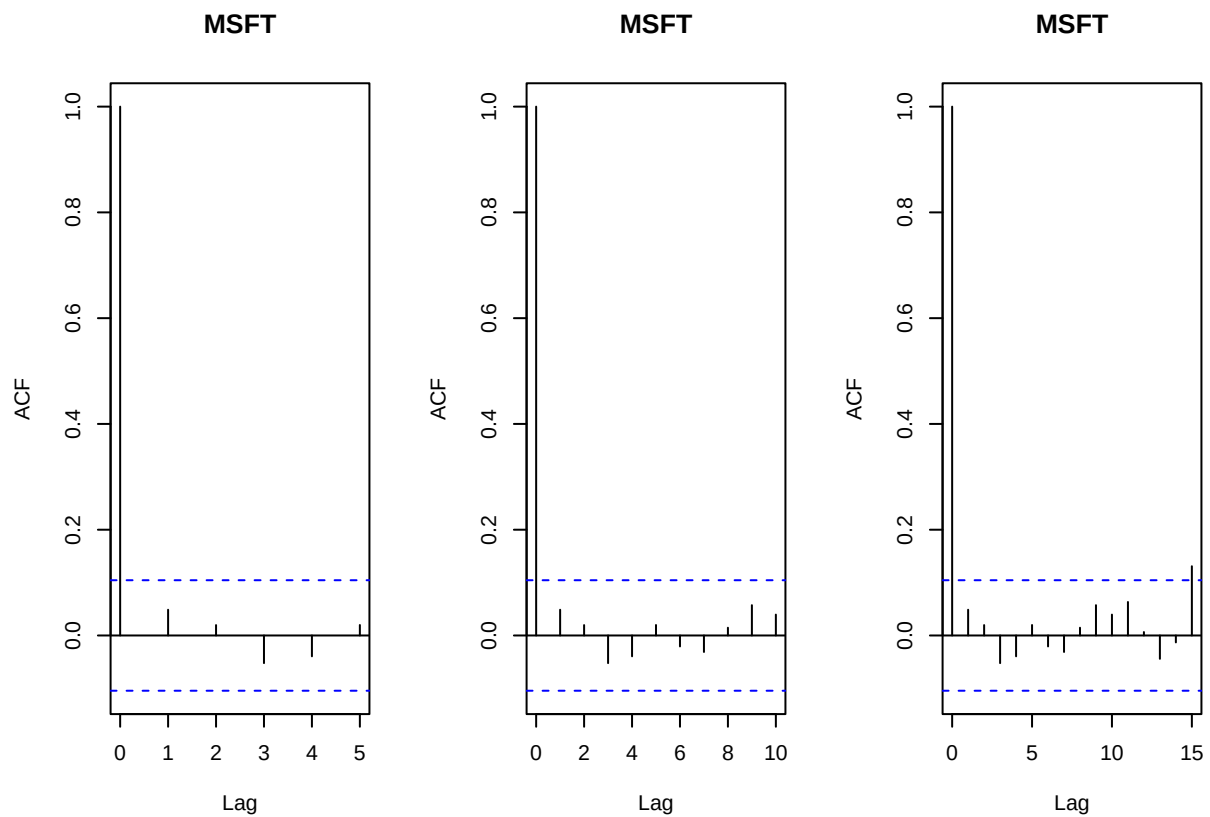
Visualizando las gráficas de autocorrelación.

```
par(mfrow = c(1, 3))

for (name in colnames(ret)){
  for (l in lags) {
    acf(
      ret[name],
      lag.max = l,
      main = cat(name, " - ACF - Lag ", l, sep = "")
    )
  }
}
```

```
## MSFT - ACF - Lag 5
```

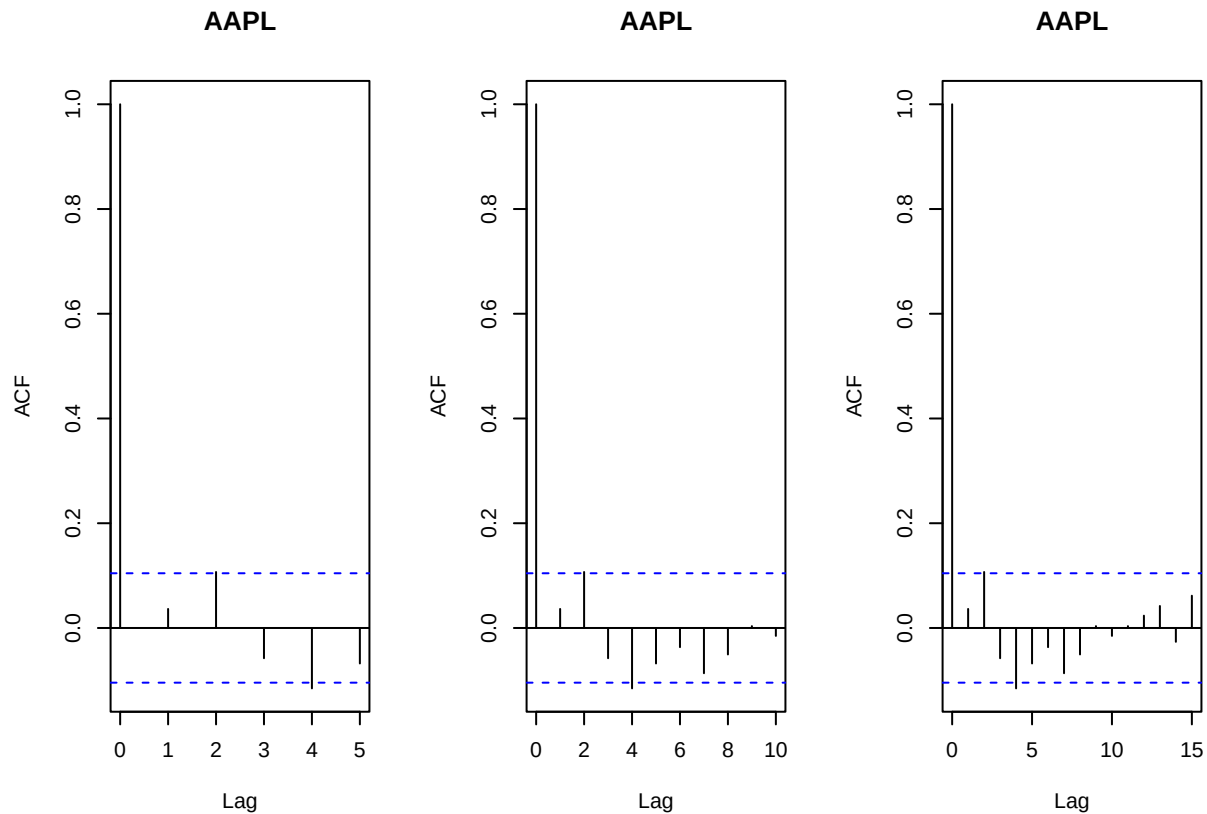
```
## MSFT - ACF - Lag 10
```



```
## MSFT - ACF - Lag 15
```

```
## AAPL - ACF - Lag 5
```

```
## AAPL - ACF - Lag 10
```



```
## AAPL - ACF - Lag 15
par(mfrow = c(1, 1))
```

Ejercicio 2

Genere un ciclo para evaluar distintos parámetros de los modelos $\text{ARMA}(m, n) - \text{GARCH}(p, q)$. Considere $0 \leq m, n \leq 4y 0 \leq p, q \leq 2$. Presente la matriz con los resultados incluyendo las pruebas estadísticas a los residuales estandarizados de cada modelo así como los parámetros (ordénela de acuerdo a mayor p-value de la prueba Ljung-Box promedio). Diga qué modelo escogería para cada una de las dos series y por qué.

Generando el espacio de hiperparámetros sobre el que vamos a iterar cada serie

```
eval_model <- function(serie){
  resultados <- data.frame()
  for (cur_m in 0:4) {
    for (cur_n in 0:4) {
      for (cur_p in 0:2) {
        for (cur_q in 0:2) {
          # Saltamos iteración inválida
          if (cur_p == 0 && cur_q == 0) { next }
          # ID iteración
          cat(
            "ARMA(", cur_m, ",", cur_n,
            ") GARCH(", cur_p, ",", cur_q, ")\n"
          )
          # Manejo de errores
          tryCatch({
```

```

# Modelo GARCH
spec <- ugarchspec(
  variance.model = list(
    model = "sGARCH",
    garchOrder = c(cur_p, cur_q)
  ),
  mean.model = list(
    armaOrder = c(cur_m, cur_n),
    include.mean = TRUE
  ),
  distribution.model = "norm"
)
print("Modelo GARCH generado correctamente")
# Ajuste
ajuste <- ugarchfit(
  spec = spec,
  data = serie,
  solver = "hybrid"
)
print("Modelo GARCH ajustado correctamente")
# Residuales
resid_std <- residuals(
  ajuste,
  standardize = TRUE
)
print("Residuales estandarizados")
# Prueba Ljung-Box sobre residuales
lb_res <- Box.test(
  resid_std,
  lag = 15,
  type = "Ljung-Box"
)
print("Pruebas Ljung-Box")
# Prueba Ljung-Box sobre residuales al cuadrado
lb_sq <- Box.test(
  resid_std^2,
  lag = 15,
  type = "Ljung-Box"
)
print("Pruebas Ljung Box sobre el cuadrado")
# Criterio de información para ev. de modelos
ic <- infocriteria(ajuste)
# Versión tabular de resultados
resultados <- rbind(
  resultados,
  data.frame(
    AR = cur_m,
    MA = cur_n,
    GARCH = cur_p,
    ARCH = cur_q,
    AIC = ic[1],
    BIC = ic[2],
    LB_Resid = lb_res$p.value,

```

```

        LB_Resid2 = lb_sq$p.value,
        LB_Promedio =
            mean(
                c(
                    lb_res$p.value,
                    lb_sq$p.value
                )
            )
        )
    )
    print("Evaluado correctamente")
}, error = function(e) {print(e)})
}
}
}
}
resultados[
    order(
        resultados$LB_Promedio,
        decreasing = TRUE
    ),
]
}

```

Ahora generamos la estimación de los modelos a lo largo del espacio de las 224 combinaciones posibles para los activos MSFT y AAPL. Esto se realiza en unas celdas no incluida en el proceso final debido a que toma demasiado espacio.

Vamos a guardar los resultados en un csv

```

write.csv(
    tab_MSFT,
    "results/MSFT_res.csv",
    row.names = FALSE
)

write.csv(
    tab_AAPL,
    "results/AAPL_res.csv",
    row.names = FALSE
)

```

Ejercicio

Ejercicio

Agradecimientos

Agradezco la enorme paciencia que me tuvieron a lo largo del semestre. Aprendí mucho más de lo que esperaba (en gran medida en contra de mi voluntad) y espero que me sea de utilidad a lo largo de mi vida profesional.